inmind.academy
by inmind.ai

Front-end/Back-end Communication – Services – RxJS – Reactive Forms

SPRING 2026

inmind.academy
by inmind.ai

Overview

- Data Types and Objects
- Front-End / Back-End communication
- Angular Services
- Observables and RxJS
- Reactive Forms

inmind.academy
by inmind.ai

Data Types and Objects

inmind.academy
by inmind.ai

Main Data Types

```
let decimal : number = 6
```

Number

```
let color : string = “blue”
```

String

```
let isDone: boolean = false
```

Boolean

```
let list: Array<number> = [1, 2, 3]
```

Array

```
let scores: { [key: string] : number } = {}

let scores: Record<string, number>() = {};

let scores = new Map<string, number>();
```

Key/value pairs

```
enum Direction {
      Up,      // 0
      Down,  // 1
      Left,     // 2
      Right   // 3
}
let go: Direction;
go = Direction.Up; // 0
```

Enum

inmind.academy
by inmind.ai

Key-Value Pairs - objects

Objects

as of ES6, the order of properties in an object is based on the order of insertion but with a catch: integer keys (e.g., "1", "2") get sorted in ascending order.

Performance: Objects are generally quick and suitable for smaller data exchanges. They aren't optimized for situations where order is paramount or where keys are frequently added or removed.

```
let user: {[key: string]: string | number} = {
  "name": "Alice",
  "age": 30,
  "3": "This is a strange property key",
  "1": "Integer keys get sorted"
};

console.log(Object.keys(user));
// Output: ['1', '3', 'name', 'age']

console.log(Object.keys(user).length);
// Output: 4
```

inmind.academy
by inmind.ai

Key-Value Pairs - maps

Maps

The order in which key-value pairs were added is maintained. You can iterate over them and maintain that order, unlike traditional objects.

Performance: Maps are optimized for scenarios involving frequent additions and deletions of key-value pairs. They are suitable for managing larger data loads and ensure constant-time complexity for data fetches and retrievals.

```
let orders = new Map();
orders.set("firstOrder", "Coffee");
orders.set("secondOrder", "Tea");
orders.set("thirdOrder", "Cake");

console.log(orders.size);
// Output: 3

orders.forEach( callbackfn: (value, key) => {
  console.log(`${key}: ${value}`);
});
// Output:
// firstOrder: Coffee
// secondOrder: Tea
// thirdOrder: Cake

console.log([...orders.keys()]);
// Output: ['firstOrder', 'secondOrder', 'thirdOrder']
```

inmind.academy
by inmind.ai

Key-Value Pairs - Records

Records

Mainly used for custom keys, limiting the possible key values

The Record type ensures that userStatus can only have keys among "online", "offline", and "away". Trying to add a "busy" key will result in a compile-time error.

```
type StatusUpdates = Record<"online" | "offline" | "away", string>;

let userStatus: StatusUpdates = {
  online: "Green",
  offline: "Grey",
  away: "Yellow"
};

// This will cause a TypeScript error:
// userStatus["busy"] = "Red";
```

inmind.academy
by inmind.ai

JSON

Data Representation Format

- Stands for **JavaScript Object Notation**
- Is often used when data is **sent from a server to a web page**
- Is a lightweight format for **storing and transporting data**
- Is "self-describing" and easy to understand
- Is used while writing JavaScript based applications that includes browser extensions and websites.
- JSON format is used for **serializing and transmitting structured data over network connection.**
- **Web services and APIs use JSON format to provide public data.**
- Can be used with modern programming languages

```
{
 "squadName": "Super hero squad",
 "homeTown": "Metro City",
 "formed": 2016,
 "secretBase": "Super tower",
 "active": true,
 "members": [
  {
   "name": "Molecule Man",
   "age": 29,
   "secretIdentity": "Dan Jukes",
   "powers": [
    "Radiation resistance",
    "Turning tiny",
    "Radiaion blast"
   ]
  },
  }
```

inmind.academy
by inmind.ai

Angular Interfaces

- The interface is a **definition of the properties and methods**
- The class that implements that interface has the actual code for each of those defined properties and methods.
- In Typescript, **you can use the interface itself as a data type.**
- Interfaces in Typescript are a development time only concept, they are not included in the final JavaScript after the build process.

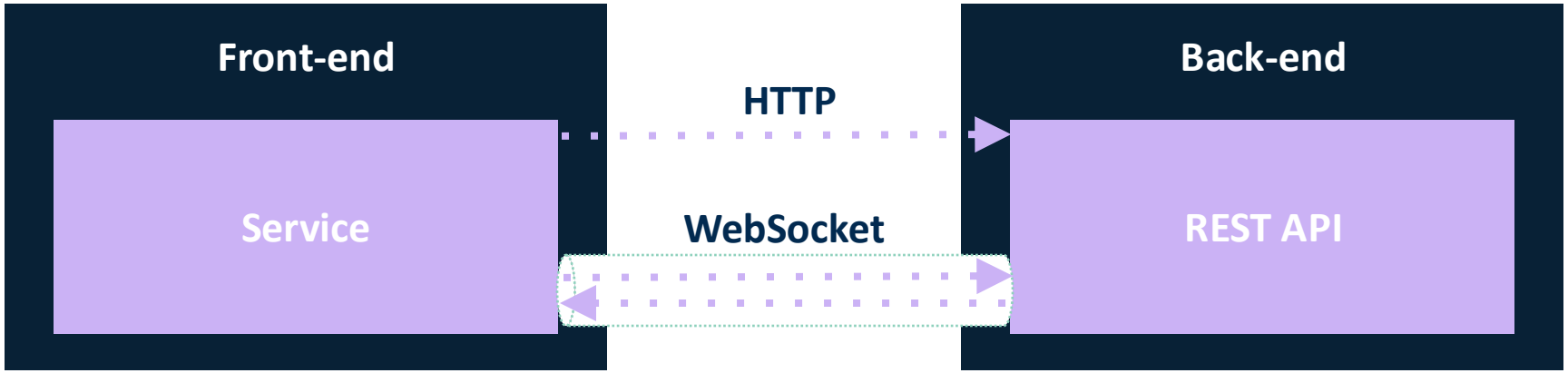
```
export interface IPerson {
   firstName: string;
   lastName: string;
   address: string;
}

/*****/
people: IPerson[] = [...]
```

inmind.academy
by inmind.ai

Front-end/Back-end communication

inmind.academy
by inmind.ai

Overview



HTTP

Front-end sends HTTP requests to the REST API (Back-end) through services
Back-end returns an HTTP response

WebSocket

Front-end initiates a two-way interactive communication session with the back-end
Data can be sent by either side (no request-response)

inmind.academy
by inmind.ai

WebSocket

Two-way communication

Both client and server can send data, there is no need for request-response (usually used to send live data from back-end to front-end)

Stringified JSON messages

All messages sent via the websocket should be stringified JSON messages can be parsed on both ends into objects when received.

Events

- **OpenEvent -** The event sent by the WebSocket object when the connection is established.
- **CloseEvent -** The event sent by the WebSocket object when the connection closes.
- **ErrorEvent -** The event sent by the WebSocket object when the connection closes unexpectedly.
- **MessageEvent -** The event sent by the WebSocket object when a message is received from the server.

inmind.academy
by inmind.ai

HTTP

The most used operations that can be done with HTTP:
GET, POST, PUT, DELETE

Request Methods

HTTP message headers are used to describe a resource, or the behavior of the server or the client.

Headers

- Form-data (file uploads)
- JSON (nested data objects)

Body

Send data via URL (encoded):
https://example.com/over/there?name=example%20name

Params

HTTP response codes indicate whether a specific HTTP request has been successfully completed. Responses are grouped in five classes:

- Informational responses (100 - 199)
- Successful responses (200 - 299)
- Redirections (300 - 399)
- Client errors (400 - 499)
- Server errors (500 – 599)

Response Codes

inmind.academy
by inmind.ai

Angular HTTP Client

- Angular provides a client HTTP API for its applications
- **HttpClient** service class in **@angular/common/http**
- Import the Angular **HttpClientModule** in the **AppModule** & inject the **HttpClient** service as a dependency of an application class
- The methods provided are asynchronous methods that send HTTP requests, and return Observables that emit the requested data when the response is received. Because the service method returns an Observable of configuration data, the component *subscribes* to the method's return value

```
configUrl = 'assets/config.json';

getConfig() {
  return this.http.get<Config>(this.configUrl);
}
```

```
showConfig() {
  this.configService.getConfig()
    .subscribe((data: Config) => this.config = {
        heroesUrl: data.heroesUrl,
        textfile:  data.textfile,
        date: data.date,
    });
}
```

inmind.academy
by inmind.ai

Angular Services

inmind.academy
by inmind.ai

Overview

- Service is a broad category encompassing **any value, function, or feature that an application needs**. A service is typically a class with a narrow, **well-defined purpose**. It should do something specific and do it well.
- Angular distinguishes components from services to **increase modularity and reusability.**
- Ideally, **a component's job is to enable only the user experience**. A component should present properties and methods for data binding to mediate between the view and the application logic. The view is what the template renders and the application logic handles the behavior

```
import { inject, Injectable } from '@angular/core';
import { AdvancedDataStore } from './advanced-data-store';

@Injectable({
  providedIn: 'root',
})
export class BasicDataStore {
  private advancedDataStore = inject(AdvancedDataStore);
  private data: string[] = [];

  addData(item: string): void {
    this.data.push(item);
  }

  getData(): string[] {
    return [...this.data, ...this.advancedDataStore.getData()];
  }
}
```

inmind.academy
by inmind.ai

Services Explained

Services in Angular are essentially objects that can be reused across different parts of an Angular application. They are commonly used for:

- **Shared Logic:** Any piece of code or logic that needs to be shared across components, directives, or other services.
- **Data Storage:** To keep data across the lifespan of the application, acting as a kind of "data memory".
- **HTTP Requests:** To handle API calls and return data to components.
- **Utility Functions**: Helper functions or utilities that can be used in many places.

inmind.academy
by inmind.ai

Injectable Decorator

- The **@Injectable() decorator defines a class as a service** in Angular and allows Angular to inject it into a component as a dependency.

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root',
})
export class HeroService {

  constructor() { }

}
```

inmind.academy
by inmind.ai

Injecting in Component

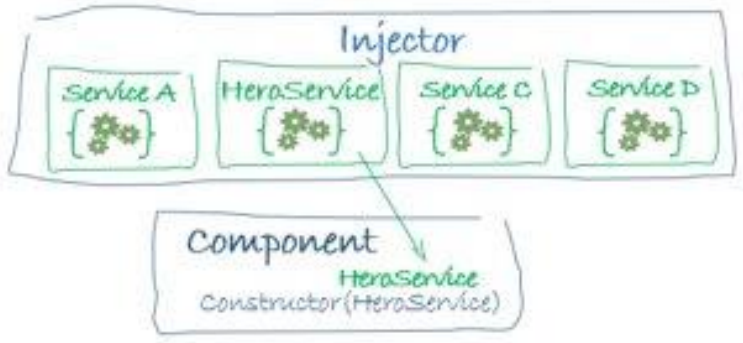
- We inject MyService into MyComponent using Angular's dependency injection system.
- When Angular creates an instance of MyComponent, it checks the component's declared dependencies (via the constructor parameters or the inject() function).
- Angular then provides an instance of MyService because it is registered with the dependency injection system using @Injectable({ providedIn: 'root' }).

```
import { Component, inject } from '@angular/core';
import { BasicDataStore } from './basic-data-store';

@Component({
  selector: 'app-example',
  template: `
    <div>
      <p>{{ dataStore.getData() }}</p>
      <button (click)="dataStore.addData('More data')">
        Add more data
      </button>
    </div>
  `
})
export class ExampleComponent {
  dataStore = inject(BasicDataStore);
}
```

inmind.academy
by inmind.ai

Dependency Injection

- Dependency injection (DI) is a core Angular mechanism that provides components and other classes with access to services and shared resources.
- Angular allows services to be injected into components, directives, and other services using its dependency injection system.
- During application bootstrap, Angular creates a root injector and additional injectors as needed at component or route level.
- An injector is responsible for creating dependencies and managing a container of instances, reusing them when possible, according to their scope.
- A provider defines how a dependency is created or obtained by the injector.
- For a service to be injectable, it must be registered with the dependency injection system, most commonly using @Injectable({ providedIn: 'root' }), where the service class itself acts as the provider.



inmind.academy
by inmind.ai

RxJS

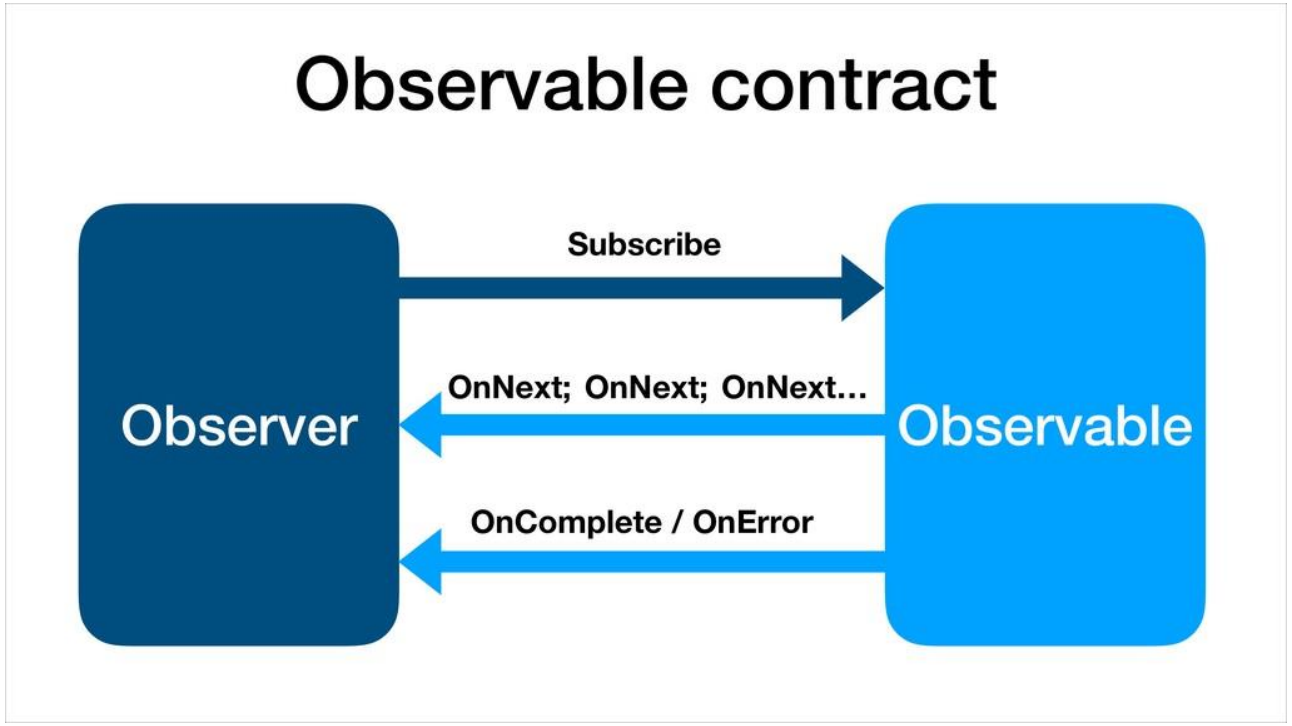
inmind.academy
by inmind.ai

Overview

- Stands for **Reactive Extension for JS**
- Library that allows us to treat data returned from asynchronous calls, as streams
- It provides one core type, the Observable, satellite types (Observers, Schedulers, Subjects) and operators to allow the handling of async events as collections
- RxJS is heavily used in Angular for async data (HTTP, routing, forms) and now coexists with Signals for local state.

Observables, Observer and the Subscription

inmind.academy
by inmind.ai



inmind.academy
by inmind.ai

Observables

An Observable is a producer of multiple values that pushes them to consumers (Observers).
Observables represent a lazy stream of values over time.

An Observable does not emit any values until a subscription is made.

Calling observable.subscribe() means *start listening for values*, either synchronously or asynchronously.

Observables emit three types of notifications:

- next
- error
- Complete

Observables can be created using creation operators or with new Observable().

Execution of an Observable begins only when it is subscribed to.

A subscription can be disposed of by calling unsubscribe() to stop receiving values.

```
1. import { Observable } from 'rxjs';
2.
3. const foo = new Observable((subscriber) => {
4.   console.log('Hello');
5.   subscriber.next(42);
6.   subscriber.next(100);
7.   subscriber.next(200);
8.   setTimeout(() => {
9.     subscriber.next(300); // happens asynchronously
10.  }, 1000);
11. });
12.
13. console.log('before');
14. foo.subscribe((x) => {
15.   console.log(x);
16. });
17. console.log('after');
```

```
"before"
"Hello"
42
100
200
"after"
300
```

inmind.academy
by inmind.ai

Observers

An Observer is a consumer of values delivered by an Observable.

An Observer is defined as a set of callbacks for each type of notification emitted by the Observable:

- next → receives a value (Number, String, Object, etc.)
- error → receives an error or exception
- complete → signals the Observable has finished, no value is sent

Each subscription to an Observable creates an independent execution for that Observer.

```
let counter = 0;

const observable = new Observable( subscribe: (subscriber: Subscriber<any>) => {
  counter++;
  subscriber.next(counter);
  subscriber.complete();
});

const observer1 = {
  next: (value: any) => console.log('Observer 1:', value)
};

const observer2 = {
  next: (value: any) => console.log('Observer 2:', value)
};

observable.subscribe(observer1); // Observer 1: 1
observable.subscribe(observer2); // Observer 2: 2
```

inmind.academy
by inmind.ai

Operators

- Used to perform operations that make the use of reactive programming more efficient: mathematics, transformation, filtering, conditional, error handling, joining.

- **Creation operators:** used to create a new observable
  For example: of(1, 2, 3) creates an observable that will emit 1, 2, and 3, one right after another.
- **Pipeable operators:** pipeable operators are functional, pure, chainable, and transform streams.

- Operators are mainly used in services for async streams.
- **Signals replace Subjects for local state**, so pipeable operators are rarely needed inside components for UI state.

```
import { of, map } from 'rxjs';

of(1, 2, 3)
  .pipe(map((x) => x * x))
  .subscribe((v) => console.log(`value: ${v}`));

// Logs:
// value: 1
// value: 4
// value: 9
```

```
@Component({ standalone: true, selector: 'app-users', template: `
  <h2>Users</h2>
  <ul>
    <li *ngFor="let user of users()">{{ user.name }}</li>
  </ul>
` })
export class UsersComponent {
  // Convert the Observable from the service into a Signal
  users = toSignal(this.userService.getUsers(), { initialValue: [] });

  constructor(private userService: UserService) {}
}
```

inmind.academy
by inmind.ai

Higher-order Observables

- A higher-order Observable is an Observable that emits other Observables.
- To work with it, we need flattening operators to convert it into a “normal” Observable.

inmind.academy
by inmind.ai

Higher-order Observables – Flattening Operators

- The **concatAll()** operator subscribes to **each "inner" Observable** that comes out of the "outer" Observable, **and copies allthe emitted values until that Observable completes, and goes on to the next one**. All of the values are in that wayconcatenated.

- Other useful flattening operators (called join operators) are:

- **mergeAll()** — subscribes to **each inner Observable as it arrives, then emits each value as it arrives**

- **switchAll()** — **subscribes to the first inner Observable when it arrives**, and emits each value as it arrives, but **whenthe next inner Observable arrives, unsubscribes to the previous one, and subscribes to the new one.**

- **exhaustAll()** — subscribes to the **first inner Observable when it arrives**, and emits each value as it arrives, **discardingall newly arriving inner Observables until that first one completes**, then waits for the next inner Observable.

inmind.academy
by inmind.ai

Higher-order Observables – Flattening Operators

- The **concatAll()** operator subscribes to **each "inner" Observable** that comes out of the "outer" Observable, **and copies allthe emitted values until that Observable completes, and goes on to the next one**. All of the values are in that wayconcatenated.

- Other useful flattening operators (called join operators) are:

- **mergeAll()** — subscribes to **each inner Observable as it arrives, then emits each value as it arrives**

- **switchAll()** — **subscribes to the first inner Observable when it arrives**, and emits each value as it arrives, but **whenthe next inner Observable arrives, unsubscribes to the previous one, and subscribes to the new one.**

- **exhaustAll()** — subscribes to the **first inner Observable when it arrives**, and emits each value as it arrives, **discardingall newly arriving inner Observables until that first one completes**, then waits for the next inner Observable.

inmind.academy
by inmind.ai

Subjects

•A Subject is like an Observable, but it can multicast to many Observers.

•Think of it as an EventEmitter: it keeps a registry of listeners and pushes values to all of them.

•There is different types of subjects:

- **BehaviorSubject:** Stores the latest valueemitted to its consumers, and whenever a newObserver subscribes, it will immediately receive the"current value".

- **ReplaySubject:** Records multiple values from theObservable execution and replays them to newsubscribers.

- **AsyncSubject:** Only the last value of the Observable execution is sent to its observers, and only when theexecution completes.

inmind.academy
by inmind.ai

Signals with RxJs

- Use RxJS in services to handle async streams
- Use Signals in components for reactive UI state
- toSignal() is the bridge between Observables and Signals
- Reduces boilerplate: no manual subscriptions or unsubscribes

```
@Injectable({ providedIn: 'root' })
export class UserService {
  constructor(private http: HttpClient) {}
  getUsers(): Observable<User[]> {
    return this.http.get<User[]>('/api/users');
  }
}

@Component({ standalone: true, template: `
  <ul>
    <li *ngFor="let user of users()">{{ user.name }}</li>
  </ul>
` })
export class UsersComponent {
  // Convert Observable to Signal
  users = toSignal(this.userService.getUsers(), { initialValue: [] });

  constructor(private userService: UserService) {}
}
```

inmind.academy
by inmind.ai

Operators

Creation Operators

- ajax
- bindCallback
- bindNodeCallback
- defer
- empty
- from
- fromEvent
- fromEventPattern
- generate
- interval
- of
- range
- throwError
- timer
- iif

Transformation Operators

- buffer
- bufferCount
- bufferTime
- bufferToggle
- bufferWhen
- concatMap
- concatMapTo
- exhaust
- exhaustMap
- expand
- groupBy
- map
- mapTo
- mergeMap
- mergeMapTo
- mergeScan
- pairwise
- partition
- pluck
- scan
- switchScan
- switchMap
- switchMapTo
- window
- windowCount
- windowTime
- windowToggle
- windowWhen

Join Creation Operators

These are Observable creation operators that also have join functionality -- emitting values of multiple source Observables.

- combineLatest
- concat
- forkJoin
- merge
- partition
- race
- zip

Mathematical and Aggregate Operators

- count
- max
- min
- reduce

inmind.academy
by inmind.ai

Operators

Utility Operators

- tap
- delay
- delayWhen
- dematerialize
- materialize
- observeOn
- subscribeOn
- timeInterval
- timestamp
- timeout
- timeoutWith
- toArray

Filtering Operators

- audit
- auditTime
- debounce
- debounceTime
- distinct
- distinctUntilChanged
- distinctUntilKeyChanged
- elementAt
- filter
- first
- ignoreElements
- Last
- sample
- sampleTime
- single
- skip
- skipLast
- skipUntil
- skipWhile
- take
- takeLast
- takeUntil
- takeWhile
- throttle
- throttleTime

Join Operators

- combineLatestAll
- concatAll
- exhaustAll
- mergeAll
- switchAll
- startWith
- withLatestFrom

Error Handling Operators

- catchError
- retry
- retryWhen

Multicasting Operators

- multicast
- publish
- publishBehavior
- publishLast
- publishReplay
- share

Conditional and Boolean Operators

- defaultIfEmpty
- every
- find
- findIndex
- isEmpty

inmind.academy
by inmind.ai

Reactive forms

inmind.academy
by inmind.ai

Overview

- Keep track of the global form state
- Know which parts of the form are valid and which are still invalid
- Properly display error messages to the user

Template-driven Forms

- Form model is created by directives (ngModel)
- Validation is triggered by events
- Suitable for simple forms

```
@Component({
  selector: 'app-template-favorite-color',
  template: `
    Favorite Color: <input type="text" [(ngModel)]="favoriteColor">
  `
})
export class FavoriteColorComponent {
  favoriteColor = '';
}
```

Reactive Forms

- Form model is created in the component class
- Validation can be triggered by events or programmatically
- Suitable for complex forms

```
@Component({
  selector: 'app-reactive-favorite-color',
  template: `
    Favorite Color: <input type="text" [formControl]="favoriteColorControl">
  `
})
export class FavoriteColorComponent {
  favoriteColorControl = new FormControl('');
}
```

inmind.academy
by inmind.ai

Form Control

- Generate a new **FormControl**

```ts
import { Component } from '@angular/core';
import { FormControl } from '@angular/forms';

@Component({
  selector: 'app-name-editor',
  templateUrl: './name-editor.component.html',
  styleUrls: ['./name-editor.component.css']
})
export class NameEditorComponent {
  name = new FormControl('');
}
```

- Register the control in the template

src/app/name-editor/name-editor.component.html

```html
<label for="name">Name: </label>
<input id="name" type="text" [formControl]="name">
```

- Display the value

```html
<p>Value: {{ name.value }}</p>
```

inmind.academy
by inmind.ai

Form Group

- Create a **FormGroup** instance

```
import { Component } from '@angular/core';
import { FormGroup, FormControl } from '@angular/forms';

@Component({
  selector: 'app-profile-editor',
  templateUrl: './profile-editor.component.html',
  styleUrls: ['./profile-editor.component.css']
})
export class ProfileEditorComponent {
  profileForm = new FormGroup({
    firstName: new FormControl(''),
    lastName: new FormControl(''),
  });
}
```

- Associate the **FormGroup** model and view

```
<form [formGroup]="profileForm">

  <label for="first-name">First Name: </label>
  <input id="first-name" type="text" formControlName="firstName">

  <label for="last-name">Last Name: </label>
  <input id="last-name" type="text" formControlName="lastName">

</form>
```

- Save form data

```
<form [formGroup]="profileForm" (ngSubmit)="onSubmit()">
```

```
onSubmit() {
  // TODO: Use EventEmitter with form value
  console.warn(this.profileForm.value);
}
```

```
<p>Complete the form to enable button.</p>
<button type="submit"
[disabled]="!profileForm.valid">Submit</button>
```

Updating the data model

`setValue()`	Set a new value for an individual control. The `setValue()` method strictly adheres to the structure of the form group and replaces the entire value for the control.
`patchValue()`	Replace any properties defined in the object that have changed in the form model.

```
updateProfile() {
  this.profileForm.patchValue({
    firstName: 'Nancy',
    address: {
      street: '123 Drew Street'
    }
  });
}
```

inmind.academy
by inmind.ai

Form Builder

- Import the **FormBuilder** class & inject the **FormBuilder** service

```
import { FormBuilder } from '@angular/forms';

constructor(private fb: FormBuilder) { }
```

- Generate form controls

```
import { Component } from '@angular/core';
import { FormBuilder } from '@angular/forms';

@Component({
  selector: 'app-profile-editor',
  templateUrl: './profile-editor.component.html',
  styleUrls: ['./profile-editor.component.css']
})
export class ProfileEditorComponent {
  profileForm = this.fb.group({
    firstName: [''],
    lastName: [''],
    address: this.fb.group({
      street: [''],
      city: [''],
      state: [''],
      zip: ['']
    }),
  });

  constructor(private fb: FormBuilder) { }
}
```

inmind.academy
by inmind.ai

Validation

- Import a validator function

```
import { Validators } from '@angular/forms';
```

- Make a field required

```
profileForm = this.fb.group({
  firstName: ['', Validators.required],
  lastName: [''],
  address: this.fb.group({
    street: [''],
    city: [''],
    state: [''],
    zip: ['']
  }),
});
```

- Display form status

```
<p>Form Status: {{ profileForm.status }}</p>
```

* You can choose to write your own validator functions, or you can use some of Angular's built-in validators.

```
class Validators {
  static min(min: number): ValidatorFn
  static max(max: number): ValidatorFn
  static required(control: AbstractControl<any, any>): ValidationErrors | null
  static requiredTrue(control: AbstractControl<any, any>): ValidationErrors | null
  static email(control: AbstractControl<any, any>): ValidationErrors | null
  static minLength(minLength: number): ValidatorFn
  static maxLength(maxLength: number): ValidatorFn
  static pattern(pattern: string | RegExp): ValidatorFn
  static nullValidator(control: AbstractControl<any, any>): ValidationErrors | null
  static compose(validators: ValidatorFn[]): ValidatorFn | null
  static composeAsync(validators: AsyncValidatorFn[]): AsyncValidatorFn | null
}
```

inmind.academy
by inmind.ai

Hands-on

Create a reactive form
Create a custom validator

inmind.academy
by inmind.ai

Homework

- Use the newly explained concepts, and the below link to the API, to continue building your project

https://fakestoreapi.com/docs

Before next session

- Create a service with at least one API call, showing the respective results in the UI. The function should expect and return defined types

- The implementation of at least one of the login and registration pages should be created/modified to use the reactive forms

inmind.academy
by inmind.ai

THANK YOU

Questions?

ADDRESS

Mkalles, main road, bld. 757, Lebanon

CONTACT

Phone +961 70 478 758
Email inmind.academy@inmind.ai
Website www.inmind.academy

SOCIAL MEDIA